

Practical - 2

Aim: To Implementation and Time analysis of linear and binary search algorithm.

1) Linear Search

Algorithm:

Linear Search Algorithm

Input: An array of n elements and a search element (key). Eg: 10, 25, 30, 45, 50 key = 30

Output: The position of search element if it is found, otherwise Element not found. Eg: Element found at position 3

Step-1 : Start

Step-2 : Create an array and enter the size of array

Step-3 : Enter array elements

Step-4 : Enter the element to be searched (key).

Step-5 : Set $i = 0$

Step-6 : compare $arr[i]$ with key

→ if $arr[i] == key$ then print "Element found at index i".

→ else increase i by 1.

Step-7 : Repeat until $i < size$.

Step-8 : if $i == size$, print "Element not found".

Step-9 : Stop

Program:

```
#include <iostream>
using namespace std;
class LinearSearch
{
public:
    int arr[100], n, key;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }

        cout << "Enter the element to search: ";
        cin >> key;
```

Name : Vemalatha Bhimireddy
Enrollment Number: 92460118162

```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void search()
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at index " << i << endl;
            break;
        }
    }

    if (i == n)
    {
        cout << "Element not found." << endl;
    }
}

};

int main()
{
    LinearSearch ls;

    ls.arrayInput();

    cout << "\nArray: ";
    ls.printArray();

    ls.search();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ linear.cpp -o linear.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\linear.exe
Enter number of elements: 5
Enter the elements: 3 5 7 8 2
Enter the element to search: 7

Array: 3 5 7 8 2
Element found at index 2
```

Time Complexity:

Linear Search Time Complexity

```
for (int i = 0; i < size; i++) — 1 + (n+1) + n
{
    if (arr[i] == key) — n
    {
        cout << "Element found at index : " << i << endl;
        break;
    }
}
if (i == size) — 1
{
    cout << "Element not found" << endl;
}
```

$T(n) = 1 + (n+1) + n + n + 1 = 3n + 3$
 $\therefore T(n) = 3n + 3$
 $T(n) = O(n)$

Best case : If element found at first position. $O(1)$
 Eg: Array : 7 2 5 9 1
 key : 7
 $T(n) = O(1)$

Average case : If element found at middle. $O(n)$
 Eg: Array: 2 5 7 9 1
 key : 7
 $T(n) = O(n)$

Worst case : If element found at last position. $O(n)$
 Eg: Array : 2 5 7 9 1
 key : 1
 $T(n) = O(n)$

Linear search Conclusion :

Thus, the Linear Search algorithm was successfully implemented and studied. It searches for an element by checking each element of the array one by one until the required element is found. It is simple and easy to implement, with a best-case time complexity of $O(1)$ and average and worst-case time complexity of $O(n)$.

2) Binary Search:

Algorithm:

Step-9 : Stop

Binary Search Algorithm

Input: A sorted array of elements & search element (key) Ex: 10, 20, 30, 40, 50
key = 40
Output: The position of search element, otherwise element not found. Ex: Element found at position 4

Step-1 : Start

Step-2 : Create an array arr[100] and variable size and key

Step-3 : Enter size of array

Step-4 : Enter array elements and print

Step-5 : Sort the array using sorting techniques and print sorted array

Step-6 : Enter the element to be searched and store in key

Step-7 : set low = 0
high = size - 1

and calculate $mid = (low + high) / 2$

Step-8 : compare key with arr[mid]

- if key == arr[mid] → Element is found
- if key < arr[mid] → set high = mid - 1
- if key > arr[mid] → set low = mid + 1
- Repeat until low <= high

Step-9 : if low > high, print Element not found.

Program :

```
#include <iostream>
using namespace std;

class BinarySearch
{
public:
    int arr[100], n, key;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the sorted elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }

        cout << "Enter the element to search: ";
        cin >> key;
    }

    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void search()
    {
        int low = 0;
        int high = n - 1;

        while (low <= high)
        {
            int mid = (low + high) / 2;

            if (arr[mid] == key)
            {
                cout << "Element found at index " << mid << endl;
                return;
            }
        }
    }
}
```

```
        else if (key < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    cout << "Element not found." << endl;
}

};

int main()
{
    BinarySearch bs;

    bs.arrayInput();

    cout << "\nArray: ";
    bs.printArray();

    bs.search();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ binary.cpp -o binary.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\binary.exe
Enter number of elements: 5
Enter the sorted elements: 10 45 67 86 78
Enter the element to search: 45

Array: 10 45 67 86 78
Element found at index 1
```

Time complexity:

Binary Search time complexity:

```

low = 0
high = n - 1
while (low <= high) — log n
{
    mid = (low + high) / 2 — 1
    if (arr[mid] == key) — 1
    {
        Element found
        return
    }
    else if (key < arr[mid]) — 1
    {
        high = mid - 1 — 1
    }
    else
    {
        low = mid + 1 — 1
    }
}

T(n) = 1 + log n (1 + 1 + 1 + 1)
T(n) = 1 + 4 log n
∴ T(n) = O(log n)

Worst case:
If the element is not present or is found after max no. of comparisons
Eg: Array: 10 20 30 40 50 60
    key: 80
    T(n) = O(log n)

Best case: If the element is found at middle position in first comparison
Eg: Array: 10 20 30 40 50
    key: 30
    T(n) = O(1)

Average case: If the element is found several comparisons
Eg: Array: 10 20 30 40 50 60 70
    key: 60
    T(n) = O(log n)

```

Binary Search Conclusion :

Thus, the Binary Search algorithm was successfully implemented and studied. It searches for an element by repeatedly dividing the sorted array into two halves and checking the middle element. It is faster than Linear Search for large datasets, with a best-case time complexity of $O(1)$ and average and worst-case time complexity of $O(\log n)$.

Conclusion :

Thus, we successfully implemented Linear Search and Binary Search and understood their working and time complexities. Linear Search checks elements one by one, while Binary Search repeatedly divides the sorted array into halves. We observed that Binary Search is more efficient for large sorted arrays, with a time complexity of $O(\log n)$ compared to $O(n)$ for Linear Search.